Table of Contents

1. Introduction

1.1 Purpose of the System

Tank Battle Game is a 2-D action game where the user controls a tank and fights against enemy tanks. The purpose of the system is to provide a simple, playable, and understandable game while showing object-oriented software design.

1.2 Design Goals

- Keep the game easy to play and understand.
- Use separate classes for different game objects.
- Keep the code maintainable for future improvements.
- Make the game run smoothly using a timer-based game loop.
- Use simple graphics so the project is easy to explain.

1.3 Trade-Offs

The game uses Java Swing instead of advanced game engines. This reduces development time and makes the project easier to understand, but it also limits advanced graphics. The game focuses more on basic functionality and object-oriented design than complex visual effects.

2. System Architecture

The system is designed in a simple layered style similar to the MVC idea. The user interface is handled by the Java window and drawing panel. The game management logic controls movement, collision, score, and game status. The model part is represented by objects such as Tank, EnemyTank, Bullet, and Wall.

Subsystem	Responsibility
User Interface	Displays the game screen, score, health, win message, and game over message.
Game Management	Controls timer, movement, shooting, collision checks, restarting, and win/loss conditions.
Model/Game Objects	Stores objects such as player tank, enemy tanks, bullets, and walls.

2.1 Subsystem Decomposition

The game is divided into three main parts. The first part is the interface that draws objects and gets keyboard input. The second part is game management that updates the game every few milliseconds. The third part is the model which contains all game objects.

2.2 Hardware/Software Mapping

- Programming language: Java
- GUI library: Java Swing
- Required software: Java Development Kit
- Input device: Keyboard
- Operating system: Any system that supports Java

2.3 Persistent Data Management

This version of the game does not use a database or saved files. All data such as score, health, enemy positions, and bullet positions are stored in memory while the game is running.

2.4 Access Control and Security

The game does not require login, internet connection, or personal data. Because of this, there are no major access control or security requirements.

3. Subsystem Services

3.1 User Interface Subsystem

- Creates the game window.
- Draws player tank, enemies, bullets, and walls.
- Displays health, score, and enemies left.
- Shows Game Over or You Win messages.

3.2 Game Management Subsystem

- Runs the game loop using Timer.
- Moves the player based on keyboard input.
- Moves enemies randomly.
- Creates bullets when player or enemies shoot.
- Checks collisions between bullets, tanks, and walls.
- Controls restart, winning, and losing.

3.3 Model Subsystem

- Tank stores common tank data and behavior.
- PlayerTank represents the user tank.
- EnemyTank represents computer-controlled tanks.
- Bullet represents shots fired by tanks.
- Wall represents obstacles.

4. Low-Level Design

4.1 Final Object Design

Class	Relationship	Target
TankBattleGame	Uses	PlayerTank, EnemyTank, Bullet, Wall
PlayerTank	Extends	Tank
EnemyTank	Extends	Tank
Tank	Creates	Bullet
Bullet	Checks collision with	Tank and Wall
Wall	Blocks	Tank and Bullet

4.2 Design Decisions and Design Patterns

The main object-oriented design decision is inheritance. PlayerTank and EnemyTank both extend the Tank class because they share common properties such as position, size, health, speed, direction, drawing, and shooting. This reduces repeated code and makes the program easier to maintain.

The game also uses a simple controller style inside the TankBattleGame class. This class controls the main game loop, updates game objects, and checks collisions. While this is not a full MVC implementation, the structure is still separated enough for a small student project.

5. Class Interfaces

Class	Important Methods
TankBattleGame	paintComponent(), actionPerformed(), movePlayer(), moveEnemies(), moveBullets(), checkCollisions(), restartGame()
Tank	draw(), shoot(), getBounds()
PlayerTank	Constructor for player tank
EnemyTank	move(), changeDirection()
Bullet	move(), draw(), getBounds()
Wall	draw(), getBounds()

6. Improvement Summary

- Enemy AI can be improved to follow the player.
- Different levels can be added.
- The game can save high scores in a file.
- Images can replace simple rectangles.
- Sound effects and background music can be added.